

TransparentGS

1 目的

当前的 3D-GS 算法对透明物体（折射效果）的重建存在明显问题，本论文提出了基于 3D-GS 算法的逆渲染管线，其主要贡献有以下三点：

- （1）提出了透明的高斯图元表示，可以重建折射现象；
- （2）提出了高斯光场探针（GaussProbe），将环境光和周围物体的贡献进行建模；
- （3）提出了基于深度的迭代探针查询算法（IterQuery），减少视差错误。

2 基础内容

2.1 3D-GS

3D-GS 是一种场景重建算法，其基本图元为 3D 高斯核。每个高斯核的参数为位置 μ 、协方差矩阵 Σ ，透明度 o 以及球谐系数 $SH(\cdot)$ 。

为了渲染图像，利用观察矩阵 $\mathbf{W}$ 将 3D 高斯核转化到相机空间，然后用局部仿射 Jacobian 矩阵 $\mathbf{J}$ 将 3D 高斯核转化到像素空间：

$$\Sigma_{2D} = \mathbf{JW}\Sigma\mathbf{W}^T\mathbf{J}^T \quad (1)$$

其中 Σ_{2D} 为投影后的 2D 高斯核的协方差矩阵。然后 3D-GS 利用体渲染的思路，对 3D 高斯核进行混合，类似 NeRF：

$$\mathbf{C} = \sum_{i=1}^N T_i \alpha_i SH_i(\mathbf{v}) \quad (2)$$

其中 N 是对于单个像素的图元数量， $\mathbf{v}$ 表示观察方向， $T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$ 为累积透射率。

2.2 渲染方程

经典 Kajiya 方程：

$$L = \int_{\Omega} L_{in}(\mathbf{x}, \omega_{in}) f(\mathbf{x}, \omega, \omega_{in}) |\omega_{in} \cdot \mathbf{n}| d\omega_{in} \quad (3)$$

其中 L_{in} 为入射光的贡献， $\mathbf{n}$ 为表面法线， ω 和 ω_{in} 为出射方向和入射方向， f 为 BSDF。

3 透明高斯图元

3D-GS 中的球谐函数很难捕捉到像镜面反射和折射这种高频信息，会导致过拟合（新视角结果偏差过大）或者非常模糊的结果，本文作者提出了基于 PBR 的透明高斯图元的表示方法来解决该问题。

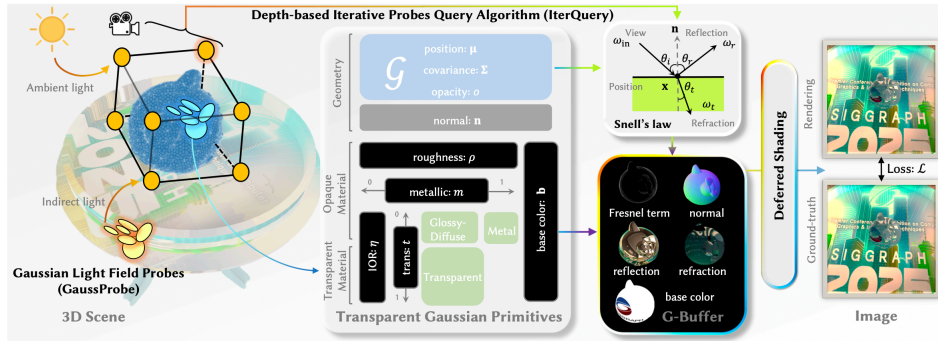


图 1: TransparentGS Pipeline

3.1 参数化表示

如图1所示，在高斯图元的几何表示中再添加法线信息 $\mathbf{n}$ ，和材质信息，包括粗糙度 ρ 和金属度 m 。对于透明物体，需要表现物体表面光的透射和折射，因此再添加透明度 t 和 IOR η 。对于最初的 3D-GS，仅依赖 opacity 去建模表示物体的不透明情况，难以计算反射和折射，所以引入参数 t ，进一步表示物体的材质状况。

3.2 反射和折射

通过两个 BSDF 来表示物体表面的反射和折射：

$$f = (1 - t)f_r + tf_t \quad (4)$$

其中 f_r 表示 BRDF， f_t 表示 BTDF。对于不透明材质， $t = 0$ ，对应 GGX 模型。对于反射中的金属部分或者粗糙度为 0 的半透明物体，将其视作镜面反射，其表示如下：

$$f_r = F \frac{\delta(\omega - \omega_r)}{|\omega_{in} \cdot \mathbf{n}|}, \quad \omega_r = 2(\omega_{in} \cdot \mathbf{n})\mathbf{n} - \omega_{in} \quad (5)$$

其中 ω_r 为反射方向， δ 为 Dirac 函数，菲涅耳项 F 由 Schlick 近似公式得到。

对于，半透明物体的 transmissive 部分，如果粗糙度为 0，将其视作镜面折射：

$$f_t = (1 - F) \frac{\delta(\omega - \omega_t)}{|\omega_{in} \cdot \mathbf{n}|} \quad (6)$$

其中 ω_t 为折射方向，由 Snell 定律计算得到。

3.3 延迟折射

3DGS 的着色策略主要分为两种：

1. 前向渲染：先进行着色，再进行 α 混合
2. 延迟渲染：先进行 α 混合，再进行着色

作者根据 DeferredGS 的延迟反射想法，提出了延迟折射的思路。将公式 (3) 和公式 (6) 可以得到透明部分的计算公式：

$$\begin{aligned} L_t &= \int_{\Omega} L_{in}(\mathbf{x}, \omega_{in})(1 - F)\delta(\omega - \omega_t)d\omega_{in} \\ &= (1 - F)L_{in}(\mathbf{x}, \omega_t) \end{aligned} \quad (7)$$

因为 α 混合是线性操作，且公式 (7) 对于法线 $\mathbf{n}$ 是非线性的，所以操作顺序对折射结果有着重大影响。如图2所示，前向折射是对光线路径上所有高斯核的着色结果进行平均；延迟着色是先计算所有高斯核加权法线：

$$\mathbf{N} = \sum_{i=1}^N T_i \alpha_i \mathbf{n}_i \quad (8)$$

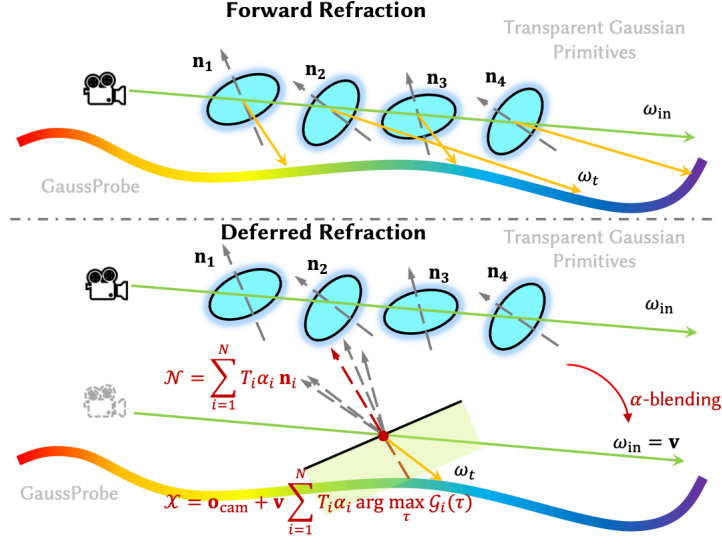


图 2: Forward Refraction and Deferred Refraction

由于反射和折射都需要考虑 secondary ray，我们同样需要计算合适的 hit point，用于计算折射和反射。如果直接使用所有高斯核的中心，会遗漏各向异性的特点，本文采取了更精确的策略来计算 hit point。对于 primary ray $\mathbf{o}_{cam} + \tau \mathbf{v}$ ，其高斯值可以表示为：

$$\mathbf{G}(\tau) = \exp\left(-\frac{1}{2}(\mathbf{o}_{cam} + \tau \mathbf{v} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{o}_{cam} + \tau \mathbf{v} - \boldsymbol{\mu})\right) \quad (9)$$

计算该函数的最大值，并以如下方式进行求和：

$$\begin{aligned} \chi &= \mathbf{o}_{cam} + \mathbf{v} \sum_{i=1}^N T_i \alpha_i \max_{\tau} \mathbf{G}_i(\tau) \\ \max_{\tau} \mathbf{G}_i(\tau) &= \frac{(\boldsymbol{\mu}_i - \mathbf{o}_{cam})^T \boldsymbol{\Sigma}^{-1} \mathbf{v}}{\mathbf{v}^T \boldsymbol{\Sigma}^{-1} \mathbf{v}} \end{aligned} \quad (10)$$

其中 χ 是加权的 hit point， $\mathbf{G}_i$ 为光线上的第 i 个高斯核，并由此计算折射光线 ω_t 和反射光线 ω_r 。

3.4 半透明物体折射

进一步考虑半透明物体对光线的吸收效果，对公式（7）进行修改：

$$L_t = (1 - F)L_{in}(\mathbf{x}, \omega_t)e^{-\sigma(\lambda)d} \quad (11)$$

其中 σ 是材质的吸收系数， λ 是光的波长， d 是光线在材质中走过的距离。指数项用对 base color 的 alpha 混合来逼近：

$$\sum_{i=1}^N T_i \alpha_i \mathbf{b}_i$$

其中 $\mathbf{b}_i$ 是每个半透明高斯核的优化参数。

4 高斯光场探针

如何表示入射光对逆渲染有着重大影响。例如，使用只有两个自由度的环境光贴图表示间接光会带来较大偏差。为了解决光线不一致的问题，本文引入了 GaussProbe。本节重要符号如图3所示。

Symbol	Definition
$\mu', \mu_x, \mu_y, \mu_z$	Position of 3D Gaussians in the camera space
φ	Optimal projection function
K, i	Number and index of Gaussian light field probes
Φ, Θ	Color and depth panorama of GaussProbe
$\mathbf{p}_i, \mathbf{d}_i$	Position and direction of i -th GaussProbe
$\mathbf{c}_i, t_i$	Color and depth of i -th GaussProbe queried by $\mathbf{d}_i$
$\mathbf{o} + t\mathbf{d}$	Queried ray (e.g., refracted or reflected ray)
$\mathbf{o} + \hat{t}\mathbf{d}$	Estimated intersection of the queried ray with Env.
w_i	Trilinear interpolation weight of i -th GaussProbe

图 3: GaussProbe 符号

4.1 烘焙 GaussProbe

图4展示了烘焙 GaussProbe 的流程：

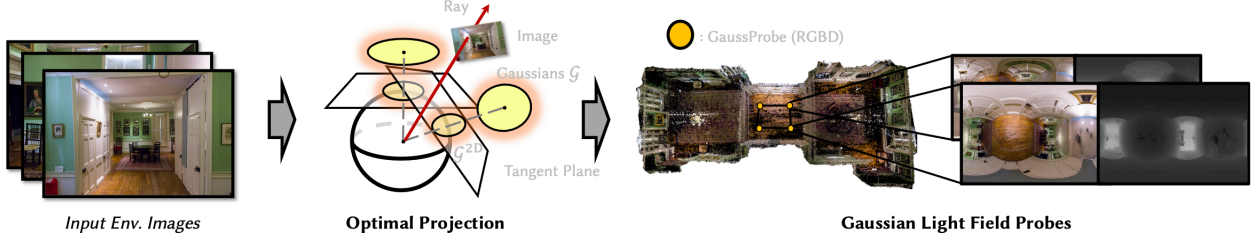


图 4: 烘焙 GaussProbe

(1) 给定去掉半透明物体的环境贴图，根据原始的 3D-GS 算法对场景进行重建；

(2) 将场景进行体素化，在半透明物体的包围盒周围摆放虚拟相机（相机位置即探针位置），对于每个虚拟相机，将高斯图元投影到对应的单位球的切线平面，生成切线平面上的高斯图元；

(3) 利用 alpha 混合绘制 360 度全景的 color 图和 depth 图，存储在 voxel 中。

我们用投影函数 ϕ 将高斯图元投影到单位球的切平面，对应的局部估计的 Jacobian 矩阵为：

$$\mathbf{J} = \frac{1}{(\mu_x^2 + \mu_y^2 + \mu_z^2)^{\frac{3}{2}}} \begin{bmatrix} \mu_y^2 + \mu_z^2 & -\mu_x\mu_y & -\mu_x\mu_z \\ -\mu_x\mu_y & \mu_x^2 + \mu_z^2 & -\mu_y\mu_z \\ -\mu_x\mu_z & -\mu_y\mu_z & \mu_x^2 + \mu_y^2 \end{bmatrix}$$

其中 $\boldsymbol{\mu}' = [\mu_x, \mu_y, \mu_z]^T$ 表示 3D 高斯图元在相机空间的位置。对于图上的像素 (u, v) ，我们投射光线并计算其与切平面的交点：

$$\mathbf{x}_{2D} = \varphi \left(\begin{bmatrix} \sin(\frac{\pi(-W+2u)}{W}) \cos(\frac{\pi(-H/2+v)}{H}) \\ \sin(\frac{\pi(-H/2+v)}{H}) \\ \cos(\frac{\pi(-H/2+v)}{H}) \cos(\frac{\pi(-W+2u)}{W}) \end{bmatrix} \right) \quad (12)$$

其中 H 和 W 对应全景图的分辨率。渲染深度全景图，深度值计算的方法为高斯核到相机位置的欧拉距离。第 i 个 probe，其位置为 $\mathbf{p}_i$ ，存储 color 全景图 Φ 和 depth 全景图 Θ ，color $\mathbf{c}_i$ 和 depth t_i 通过方向 $\mathbf{d}_i$ 计算得到：

$$\mathbf{c}_i, t_i = \Phi(\mathbf{p}_i, \mathbf{d}_i), \Theta(\mathbf{p}_i, \mathbf{d}_i) \quad (13)$$

4.2 基于深度的 probe 迭代查询

当计算反射或折射的时候，需要根据给定光线对 probe 进行查询。记 $\mathbf{o}$ 和 $\mathbf{d}$ 分别为查询光线的初始位置和方向， $\mathbf{p}_i$ 和 $\mathbf{d}_i$ 为第 i 个 probe 对应的位置和方向。如图5中 (d) 的 naive 结果所示，如果对每个 probe 直接使用光线的方向 $\mathbf{d}$ 进行查询，那么会造成结果过度模糊，重建结果较差。作者提出了基于深度的 probe 迭代查询 (IterQuery) 算法，其算法流程为：

- (1) 将每个 probe 对应的方向初始化为光线方向：

$$\mathbf{d}_i = \mathbf{d} \quad (14)$$

- (2) 每个 probe 对深度图进行采样，获得交点 $\mathbf{p}_i + t_i \mathbf{d}_i$ ：

$$t_i = \Theta(\mathbf{p}_i, \mathbf{d}_i) \quad (15)$$

- (3) 对 probe 的 depth map 进行三线性插值，并投影到查询光线上：

$$\hat{t} = \sum_{i=1}^K \omega_i ((\mathbf{p}_i + t_i \mathbf{d}_i - \mathbf{o}) \cdot \mathbf{d}) \quad (16)$$

其中 ω_i 为插值权重。然后依如下公式更新 $\mathbf{d}_i$ ：

$$\mathbf{d}_i := \frac{\mathbf{o} + \hat{t} \mathbf{d} - \mathbf{o}}{\|\mathbf{o} + \hat{t} \mathbf{d} - \mathbf{o}\|} \quad (17)$$

- (4) 重复前面的步骤，直到 $\hat{t}$ 收敛，示意图见图5，伪代码见图4.2。

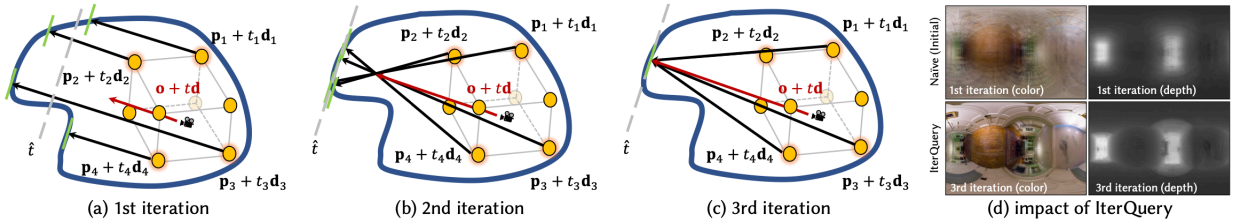


图 5: IterQuery 算法

```

1 def iter_query(probes, ray, total_iter):
2     o, d = ray[0, :3], ray[1, :3]
3     # calculate the parallax relative to the virtual camera
4     delta_depth = dot(o - probes.pos, d)
5     d_i = d # initialize the directions of probes, Eq. 15
6     while total_iter > 0:
7         # Eq. 16 and the green lines in Fig. 5
8         depth = probes.depth_map.texture(d_i)
9         cos_depth = depth * dot(d, d_i) + delta_depth
10        # Eq. 17 and the gray dashed lines in Fig. 5
11        t_hat = tri_lerp(cos_depth, o, probes.pos)
12        d_i = normalize(o + t_hat * d - probes.pos) # Eq. 18
13        if abs(depth - probes.depth_map.texture(d_i)) <= ε:
14            break # exit if converged in a certain region
15        total_iter -= 1
16        color = probes.color_map.texture(d_i)
17        color = tri_lerp(color, o, probes.pos)
18    return color

```

Listing 1: Pseudocode for our IterQuery algorithm.

IterQuery 算法的时间复杂度为 $O(TKQ)$ ，其中 T, K, Q 分别对应总的迭代次数，probe 的数量，查询光线的数量。

5 多阶段重建

对于法线正则项：

$$\mathcal{L}_{normal} = 1 - \mathcal{N} \cdot \hat{\mathcal{N}}_D \quad (18)$$

其中 $\mathcal{N}$ 为渲染的法线图， $\hat{\mathcal{N}}_D$ 为渲染的深度图的梯度。

完整的损失函数为：

$$\mathcal{L} = (1 - \lambda_1)\mathcal{L}_1 + \lambda_1\mathcal{L}_{D-SSIM} + \lambda_2\mathcal{L}_{normal} + \lambda_3\mathcal{L}_{mask} \quad (19)$$

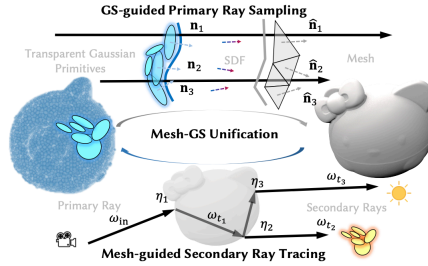


图 6: Mesh-GS Unification

为了处理折射效果的一致性，作者提出了半透明高斯图元和网格相结合的处理方法，如图6所示。对高斯图元进行采样，利用渲染的法线图生成

SDF, 然后用 SDF 生成 mesh。mesh 进行次级光线的 trace, 在 GaussProbe 的流程中使用。

6 参考

TransparentGS

DeferredGS